

206. Reverse Linked List

Solved ✓

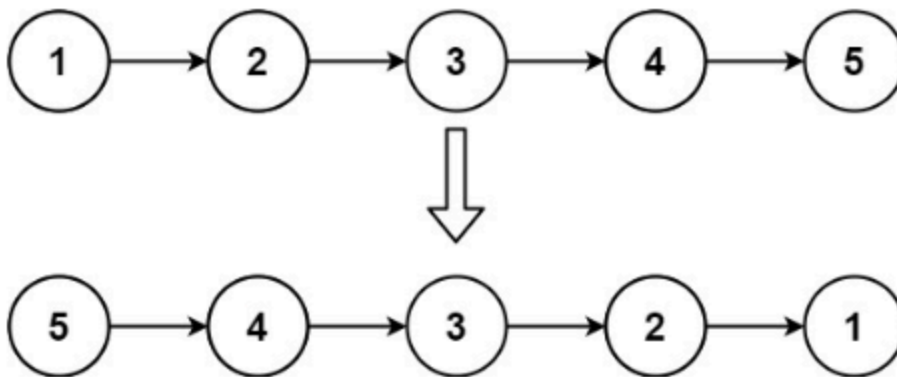
Easy

Topics

Companies

Given the `head` of a singly linked list, reverse the list, and return *the reversed list*.

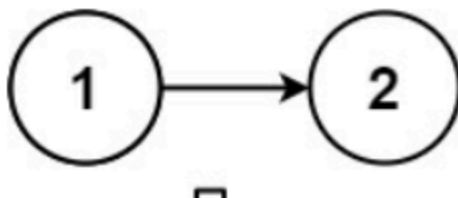
Example 1:



Input: `head = [1,2,3,4,5]`

Output: `[5,4,3,2,1]`

Example 2:



[leetcode](https://leetcode.com/problems/reverse-linked-list/)

```

class Solution {
    public ListNode reverseList(ListNode head) {
        ListNode curr = head;
        ListNode prev = null;
        ListNode next = null;
        while(curr!=null){
            next = curr.next;
            curr.next = prev;
            prev = curr;
            curr=next;
        }
        return prev;//here prev will be new head of our list
    }
}

```

```

1 class Solution {
2     public ListNode reverseList(ListNode head) {
3         if(head ==null || head.next==null)return head;
4         ListNode newHead = reverseList(head.next);
5         head.next.next = head;
6         head.next = null;
7         return newHead;
8     }
9 }

```

recursive soln [chatgpt](#)

876. Middle of the Linked List

Solved ✓

Easy

Topics

Companies

Given the `head` of a singly linked list, return *the middle node of the linked list*.

If there are two middle nodes, return **the second middle node**.

Example 1:



Input: `head = [1,2,3,4,5]`

Output: `[3,4,5]`

Explanation: The middle node of the list is node 3.

Example 2:



Input: `head = [1,2,3,4,5,6]`

[leetcode](https://leetcode.com/problems/middle-of-the-linked-list/)

21. Merge Two Sorted Lists

Solved ✓

Easy

Topics

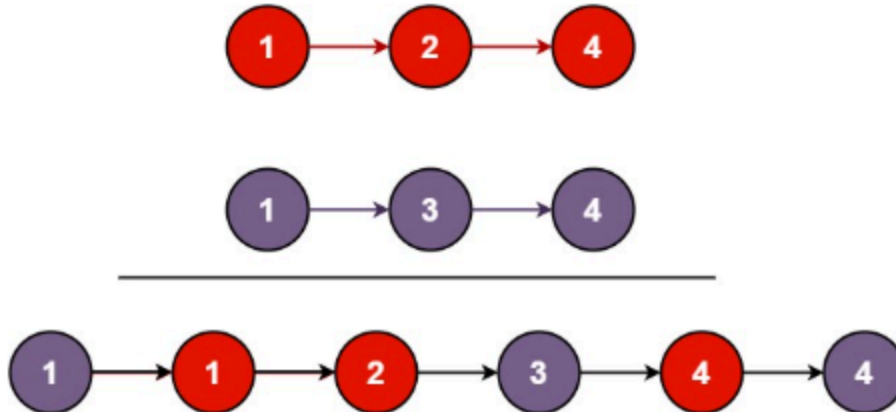
Companies

You are given the heads of two sorted linked lists `list1` and `list2`.

Merge the two lists into one **sorted** list. The list should be made by splicing together the nodes of the first two lists.

Return *the head of the merged linked list*.

Example 1:



Input: `list1 = [1,2,4]`, `list2 = [1,3,4]`

Output: `[1,1,2,3,4,4]`

[leetcode](#) [chatgpt](#)

DescriptionEditorialSolutions

2. Add Two Numbers

MediumTopicsCompanies

You are given two **non-empty** linked lists representing two non-negative integers. The digits are stored in **reverse order**, and each of their nodes contains a single digit. Add the two numbers and return the sum as a linked list.

You may assume the two numbers do not contain any leading zero, except the number 0 itself.

Example 1:

2

4

3

5

6

4

7

0

8

35.8K1K1188 Online

</> Code

JavaAuto

```
1 /**
2  * Definition for singly-linked list.
3  * public class ListNode {
4  *     int val;
5  *     ListNode next;
6  *     ListNode() {}
7  *     ListNode(int val) { this.val = val; }
8  *     ListNode(int val, ListNode next) { this.val = val; this.
9  *         next = next; }
10  * }
11  */
12 class Solution {
13     public ListNode addTwoNumbers(ListNode l1, ListNode l2) {
14     }
15 }
```

SavedLn 1, Col 1

☒ Testcase > Test Result

You must run your code first

leetcode

◆ Key Idea (Logic Thinking)

This is just like **manual addition**:

1. Add digits **one by one**
2. Keep track of **carry**
3. Move forward in both lists
4. If carry remains at the end → add a new node

```
1 class Solution {
2     public ListNode addTwoNumbers(ListNode l1, ListNode l2) {
3         int carry = 0;
4         ListNode dummy = new ListNode(100);
5         ListNode curr = dummy;
6         while(l1!=null || l2!=null){
7             int sum = carry;
8             if(l1!=null){
9                 sum+=l1.val;
0                 l1 = l1.next;
1             }
2             if(l2!=null){
3                 sum+=l2.val;
4                 l2 = l2.next;
5             }
6             carry = sum/10;
7             int digit = sum%10;
8             //adding this sum node
9             curr.next = new ListNode(digit);
0             curr = curr.next;
1             //if carry still left after all the list become null
2             if(carry>0){
3                 curr.next = new ListNode(carry);
4             }
5         }
6         return dummy.next;
7     }
```

DescriptionEditorialSolutions

237. Delete Node in a Linked List

Solved

MediumTopicsCompanies

There is a singly-linked list `head` and we want to delete a node `node` in it.

You are given the node to be deleted `node`. You will **not be given access** to the first node of `head`.

All the values of the linked list are **unique**, and it is guaranteed that the given node `node` is not the last node in the linked list.

Delete the given node. Note that by deleting the node, we do not mean removing it from memory. We mean:

- The value of the given node should not exist in the linked list.
- The number of nodes in the linked list should decrease by one.
- All the values before `node` should be in the same order.
- All the values after `node` should be in the same order.

6.2K54785 Online

Code

JavaAuto

```
1 /**
2  * Definit
3  * public
4  *     int
5  *     Lis
6  *     Lis
7  * }
8  */
9 class Solu
10     public
11         nc
12         nc
13     }
14 }
```

Saved

☒ Testcase | >_1

[leetcode](https://leetcode.com/problems/delete-node-in-a-linked-list/)

Core Idea (Logic)

Since we cannot access the previous node,
we cannot delete this node directly.

 Instead:

1. Copy the **value of the next node** into current node
2. Remove the **next node** by skipping it

Effectively, the given node **disappears from the list**

160. Intersection of Two Linked Lists

Solved ✓

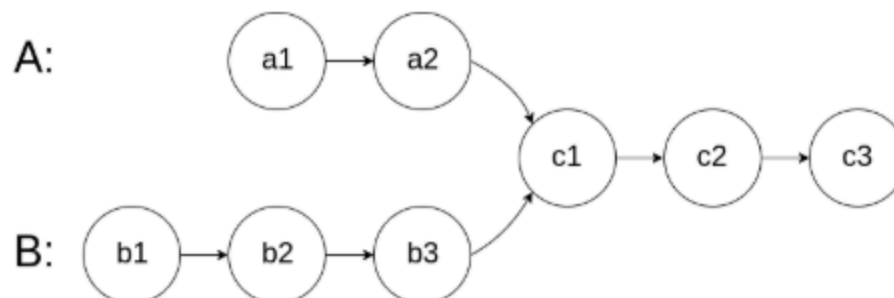
Easy

Topics

Companies

Given the heads of two singly linked-lists `headA` and `headB`, return *the node at which the two lists intersect*. If the two linked lists have no intersection at all, return `null`.

For example, the following two linked lists begin to intersect at node `c1`:



The test cases are generated such that there are no cycles anywhere in the entire linked structure.

Note that the linked lists must **retain their original structure** after the function returns.

Custom Judge:

The inputs to the **judge** are given as follows (your program is **not** given these inputs):

```

1  | */
2  public class Solution {
3      public boolean hasCycle(ListNode head) {
4          ListNode slow = head;
5          ListNode fast = head;
6          if(head==null){
7              return false;
8          }
9          //one elemnt case
10         if(fast.next == null){
11             return false;
12         }
13         while(fast!=null){
14             if(slow==null)return false;
15             slow = slow.next;
16             if(fast.next ==null)return false;
17             fast = fast.next.next;
18             if(slow==fast){
19                 return true;
20             }
21         }
22         return false;//jb loop oga nhi tb hi fast can be null
23         then while loop can end
24     }
25 }

```

25. Reverse Nodes in k-Group

Hard

Topics

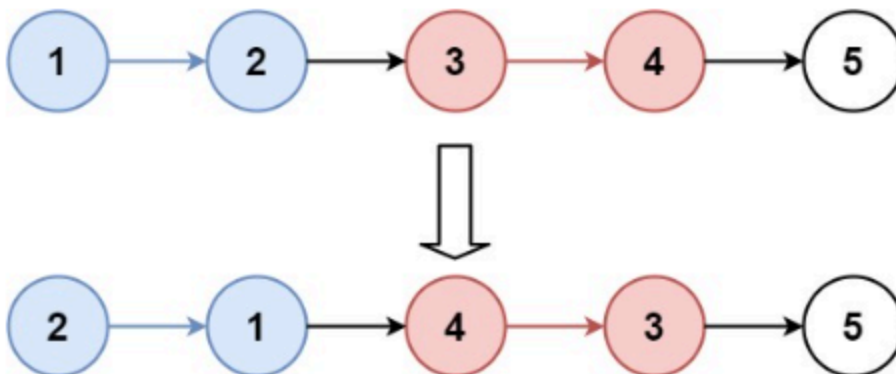
Companies

Given the `head` of a linked list, reverse the nodes of the list `k` at a time, and return *the modified list*.

`k` is a positive integer and is less than or equal to the length of the linked list. If the number of nodes is not a multiple of `k` then left-out nodes, in the end, should remain as it is.

You may not alter the values in the list's nodes, only nodes themselves may be changed.

Example 1:



Input: `head = [1,2,3,4,5]`, `k = 2`

Output: `[2,1,4,3,5]`

15.4K

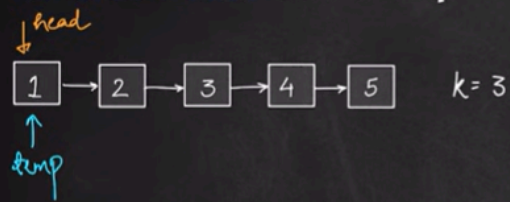


309



222 Online

Reverse Nodes in k-Group



```
cnt = 0
while (cnt < k) {
    if (temp == NULL)
        return head;
    temp = temp -> next;
}
```

- ① check if k nodes exist
- ② recursively rest of the LL
- ③ reverse the current group



```

3  */
1  class Solution {
2      public ListNode reverseKGroup(ListNode head, int k) {
3          int count =0;
4          //checking k nodes exists or not
5          ListNode temp = head;
6          while(count<k){
7              if(temp==null)return head;
8              temp = temp.next;
9              count++;
10         }
11         //temp is pointing to (k+1)th node
12         //recursive call
13         ListNode prevNode = reverseKGroup(temp,k);
14         //reversing other
15         count =0;
16         temp =head;
17         while(count<k){
18             ListNode next =temp.next;
19             temp.next = prevNode;
20             prevNode = temp;
21             temp = next;
22             count++;
23         }
24         return prevNode;
25     }
26 }

```

[chatgpt](#) [leetcode](#)

A **palindrome** is something that **reads the same forward and backward**.

Examples

1. Words

- **madam** → madam
- **racecar** → racecar
- **level**

2. Numbers

- **121** → 121
- **1331** → 1331
- **999**

3. Sentences (ignoring spaces, punctuation, and case)

- **"A man, a plan, a canal: Panama"**
- **"Was it a rat I saw?"**

Simple definition



👉 If reversing it gives the **same result**, it is a **palindrome**.

234. Palindrome Linked List

Solved ✓

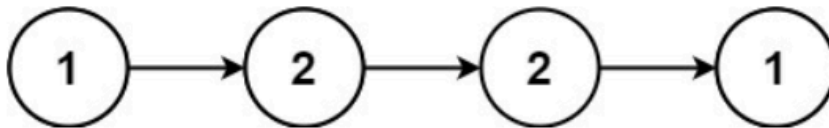
Easy

Topics

Companies

Given the `head` of a singly linked list, return `true` if it is a *palindrome* or `false` otherwise.

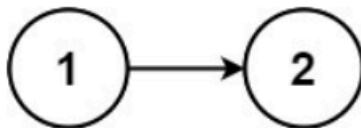
Example 1:



Input: `head = [1,2,2,1]`

Output: `true`

Example 2:



Input: `head = [1,2]`

Output: `false`

Constraints:

👍 18K 🗨 374 | ☆ 📄 ?

● 179 Online

```

11 class Solution {
12     public static ListNode reverse(ListNode head){
13         ListNode prev = null;
14         ListNode after = null;
15         ListNode current = head;
16         while(current!=null){
17             after = current.next;
18             current.next = prev;
19             prev = current;
20             current = after;
21         }
22         return prev;
23     }
24     public boolean isPalindrome(ListNode head) {
25         ListNode slow = head;
26         ListNode fast = head;
27         //to find left middle
28         while(fast.next!=null &&fast.next.next!=null){
29             fast = fast.next.next;
30             slow = slow.next;
31         }
32         //left middle = slow //now we have to reverse the elemnt after left middle or middle one in case of
odd
33         ListNode temp = reverse(slow.next);
34         slow.next = temp;
35         ListNode p1 = head;
36         ListNode p2 = slow.next;
37         while(p2!=null){

```

Saved

Ln 32, Col 112

```

22         return prev;
23     }
24     public boolean isPalindrome(ListNode head) {
25         ListNode slow = head;
26         ListNode fast = head;
27         //to find left middle
28         while(fast.next!=null &&fast.next.next!=null){
29             fast = fast.next.next;
30             slow = slow.next;
31         }
32         //left middle = slow //now we have to reverse the elemnt after left middle or middle one in case of
odd
33         ListNode temp = reverse(slow.next);
34         slow.next = temp;
35         ListNode p1 = head;
36         ListNode p2 = slow.next;
37         while(p2!=null){
38             if(p1.val!=p2.val)return false;
39             p1 = p1.next;
40             p2 = p2.next;
41         }
42         return true;
43     }
44 }

```


leetcode

Logic is 1st find mid element which we can find using two pointers slow and fast fast = fast.next.next and slow = slow.next you will observe that when fast.next==null or fast.next.next ==null then slow will be at midd element or 1st mid element Then reverse the list after mid Then take two pointer like temp1=head and temp2=head of reverse list and compare the values if value are not equal then return false

142. Linked List Cycle II

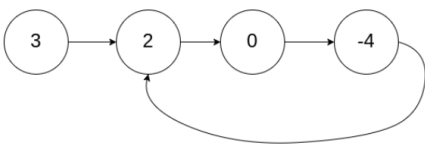
Medium Topics Companies

Given the `head` of a linked list, return the node where the cycle begins. If there is no cycle, return `null`.

There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the `next` pointer. Internally, `pos` is used to denote the index of the node that tail's `next` pointer is connected to (0-indexed). It is `-1` if there is no cycle. **Note that** `pos` is not passed as a parameter.

Do not modify the linked list.

Example 1:



```
graph LR; 3((3)) --> 2((2)); 2 --> 0((0)); 0 --> -4((-4)); -4 --> 2;
```

Input: `head = [3,2,0,-4], pos = 1`
Output: tail connects to node index 1
Explanation: There is a cycle in the linked list, where tail connects to node index 1.

14.9K 243 128 Online

```
Java Auto
9  * }
10 * }
11 */
12 public class Solution {
13     public ListNode detectCycle(ListNode head) {
14         ListNode fast = head;
15         ListNode slow = head;
16         if(head==null)return null;//no element
17         if(head.next==null)return null;//one element
18         while(fast!=null){
19             if(slow==null)return null;
20             slow = slow.next;
21             if(fast.next==null)return null;
22             fast = fast.next.next;
23             if(fast==slow)break;
24         }
25         if(fast==null)return null;
26         ListNode temp = head;
27         while(temp!=slow){
28             temp = temp.next;
29             slow = slow.next;
30         }
31         return slow;
32     }
33 }
```

Saved

Testcase Test Result

How to Explain This in an Interview

"First I use slow and fast pointers to detect a cycle.

If they meet, I reset one pointer to the head and move both one step at a time.

Due to the mathematical relationship between distances in the cycle, they meet exactly at the starting node of the cycle."

chatgpt

Flattening a Linked List



Problem Statement: Given a linked list containing 'N' head nodes where every node in the linked list contains two pointers:

- 'Next' points to the next node in the list
- 'Child' pointer to a linked list where the current node is the head

Each of these child linked lists is in sorted order and connected by a 'child' pointer. Your task is to flatten this linked list such that all nodes appear in a single layer or level in a 'sorted order'.

Examples



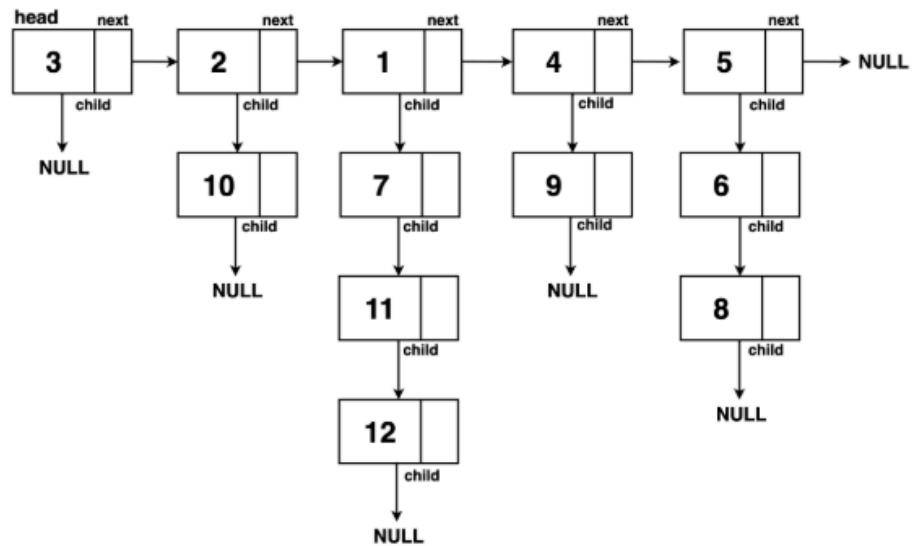
Brute Force



Optimal Approach



Input:



Output: 1 2 3 4 5 6 7 8 9 10 11 12

Brute Force

Algorithm

- Initialize an array to store node values and traverse the list, collecting values from both top-level and child nodes.
- Sort the array to arrange the values in ascending order.
- Create and return a new linked list from the sorted array.

```

1- import java.util.*;
2
3 // Node definition for Linked List with child pointers
4 class ListNode {
5     int val;
6     ListNode next;
7     ListNode child;
8
9     // Only single-value constructor
10    ListNode(int val) {
11        this.val = val;
12    }
13 }
14
15
16 class Solution {
17     // Function to convert an array list to a Linked List (using child pointers)
18     private ListNode convertArrToLinkedList(List<Integer> arr) {
19         ListNode dummyNode = new ListNode(-1);
20         ListNode temp = dummyNode;
21
22         for (int val : arr) {
23             temp.child = new ListNode(val);
24             temp = temp.child;
25         }
26         return dummyNode.child;
27     }
28
29     // Function to flatten a Linked List with child pointers
30     public ListNode flattenLinkedList(ListNode head) {
31         List<Integer> arr = new ArrayList<>();
32
33         // Collect all values from the multi-level list
34         while (head != null) {
35             ListNode t2 = head;
36             while (t2 != null) {
37                 arr.add(t2.val);
38                 t2 = t2.child;
39             }
40             head = head.next;
41         }
42
43         // Sort the values
44         Collections.sort(arr);
45
46         // Convert the sorted values back into a single-level linked list
47         return convertArrToLinkedList(arr);
48     }
49 }
50
51 class Main {
52     // Print the flattened linked list
53     public static void printLinkedList(ListNode head) {
54         while (head != null) {
55             System.out.print(head.val + " ");
56             head = head.child;
57         }
58         System.out.println();
59     }
60
61     // Print the original multi-level linked list
62     public static void printOriginalLinkedList(ListNode head, int depth) {
63         while (head != null) {
64             System.out.print(head.val);
65             if (head.child != null) {

```

```

28
29 // Function to flatten a Linked List with child pointers
30 public ListNode flattenLinkedList(ListNode head) {
31     List<Integer> arr = new ArrayList<>();
32
33     // Collect all values from the multi-level list
34     while (head != null) {
35         ListNode t2 = head;
36         while (t2 != null) {
37             arr.add(t2.val);
38             t2 = t2.child;
39         }
40         head = head.next;
41     }
42
43     // Sort the values
44     Collections.sort(arr);
45
46     // Convert the sorted values back into a single-level linked list
47     return convertArrToLinkedList(arr);
48 }
49
50
51 class Main {
52     // Print the flattened linked list
53     public static void printLinkedList(ListNode head) {
54         while (head != null) {
55             System.out.print(head.val + " ");
56             head = head.child;
57         }
58         System.out.println();
59     }
60
61     // Print the original multi-level linked list
62     public static void printOriginalLinkedList(ListNode head, int depth) {
63         while (head != null) {
64             System.out.print(head.val);
65             if (head.child != null) {

```

```

52 // Print the flattened Linked List
53 public static void printLinkedList(ListNode head) {
54     while (head != null) {
55         System.out.print(head.val + " ");
56         head = head.child;
57     }
58     System.out.println();
59 }
60
61 // Print the original multi-level Linked List
62 public static void printOriginalLinkedList(ListNode head, int depth) {
63     while (head != null) {
64         System.out.print(head.val);
65         if (head.child != null) {
66             System.out.print(" -> ");
67             printOriginalLinkedList(head.child, depth + 1);
68         }
69         if (head.next != null) {
70             System.out.println();
71             for (int i = 0; i < depth; ++i) {
72                 System.out.print("| ");
73             }
74         }
75         head = head.next;
76     }
77 }
78
79 public static void main(String[] args) {
80     // Create Linked List with child pointers using only single-value constructor
81     ListNode head = new ListNode(5);
82     head.child = new ListNode(14);
83
84     head.next = new ListNode(10);
85     head.next.child = new ListNode(4);
86
87     head.next.next = new ListNode(12);
88     head.next.next.child = new ListNode(20);

```

```

9 public static void main(String[] args) {
10     // Create Linked List with child pointers using only single-value constructor
11     ListNode head = new ListNode(5);
12     head.child = new ListNode(14);
13
14     head.next = new ListNode(10);
15     head.next.child = new ListNode(4);
16
17     head.next.next = new ListNode(12);
18     head.next.next.child = new ListNode(20);
19     head.next.next.child.child = new ListNode(13);
20
21     head.next.next.next = new ListNode(7);
22     head.next.next.next.child = new ListNode(17);
23
24     // Print original list
25     System.out.println("Original linked list:");
26     printOriginalLinkedList(head, 0);
27
28     // Flatten and print
29     Solution sol = new Solution();
30     ListNode flattened = sol.flattenLinkedList(head);
31     System.out.print("\nFlattened linked list: ");
32     printLinkedList(flattened);
33 }

```

Original linked list:

5 -> 14

10 -> 4

12 -> 20 -> 13

7 -> 17

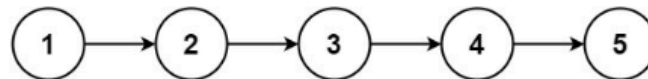
Flattened linked list: 4 5 7 10 12 13 14 17 20

61. Rotate List

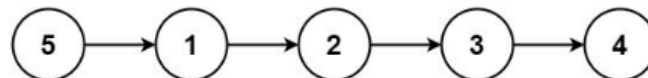
Medium Topics Companies

Given the `head` of a linked list, rotate the list to the right by `k` places.

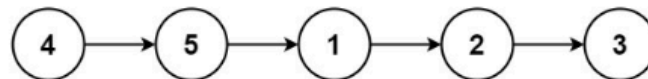
Example 1:



rotate 1



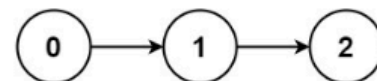
rotate 2



Input: head = [1,2,3,4,5], k = 2

Output: [4,5,1,2,3]

Example 2:



```

8 | *      ListNode(int val, ListNode next) { this.val = val;
   | this.next = next; }
9 | * }
0 | */
1 | class Solution {
2 |     public ListNode rotateRight(ListNode head, int k) {
3 | if(head==null || head.next==null)return head;
4 |
5 |         for(int i=1;i<=k;i++){
6 |             ListNode temp = head;
7 | while(temp.next.next!=null){
8 |     temp = temp.next;
9 | }
0 | ListNode after = temp.next;//jisko shift krna hai
1 | temp.next =null;
2 | after.next = head;
3 | head = after;
4 |
5 |     }
6 |     return head;
7 | }
8 | }

```

naive approach

Complexity

- Time: $O(n \times k)$ ❌
- Space: $O(1)$ ✅

This will still TLE on LeetCode.

optimal solution

Eg if no of node is like 5 and $k=7$ so if you rotate it 7 times it will be equivalent to rotation when $k=2$ i.e $k \% \text{length}$

Key Observation

Rotating right by `k` is same as:

SCSS

Rotate right by $(k \% \text{length})$

Optimal Logic ($O(n)$)

1. Find length
2. Connect tail $\rightarrow$ head (make circular list)
3. Break circle at correct point


```

10 /
11 class Solution {
12     public ListNode rotateRight(ListNode head, int k) {
13         if(head==null || head.next==null || k==0) return head;
14         ListNode temp = head;
15         int length =1; //it will store no of nodes
16         while(temp.next!=null){
17             temp=temp.next;
18             length++;
19         }
20         //making circular list
21         temp.next = head;
22
23         ListNode newtail = head;
24         k = k%length;
25         int distance = length-k; //to travel towards new head
26         for(int i=1; i<distance; i++){
27             //jis node ko move krna hai usse ek piche ye aake loop khtm ho jaega
28             newtail = newtail.next;
29         }
30         ListNode newhead = newtail.next;
31         newtail.next = null;
32         return newhead;
33     }
34 }

```

7 What to say in an interview 🗣️

“Initially I thought of rotating one step at a time, but that leads to $O(n \times k)$. So I optimized it by converting the list into a circular linked list and breaking it at the correct position using modulo.”

Time Complexity:

$O(n)$

- One traversal to find the length of the list → $O(n)$
- One traversal to find the new tail → $O(n)$
- Total = $O(n)$

Even though there are two loops, they are **sequential**, not nested.

Space Complexity:

$O(1)$

- No extra data structures used
 - Only a few pointer variables
-

138. Copy List with Random Pointer

Solved 

Medium

 Topics

 Companies

 Hint

A linked list of length `n` is given such that each node contains an additional random pointer, which could point to any node in the list, or `null`.

Construct a **deep copy** of the list. The deep copy should consist of exactly `n` **brand new** nodes, where each new node has its value set to the value of its corresponding original node. Both the `next` and `random` pointer of the new nodes should point to new nodes in the copied list such that the pointers in the original list and copied list represent the same list state. **None of the pointers in the new list should point to nodes in the original list.**

For example, if there are two nodes `X` and `Y` in the original list, where `X.random --> Y`, then for the corresponding two nodes `x` and `y` in the copied list, `x.random --> y`.

Return *the head of the copied linked list*.

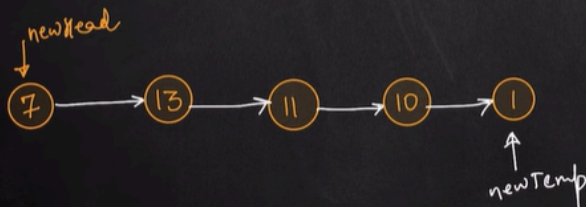
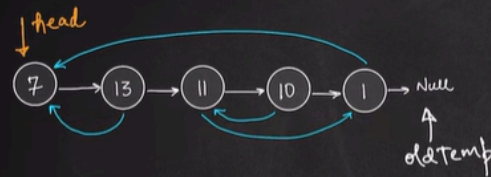
The linked list is represented in the input/output as a list of `n` nodes. Each node is represented as a pair of `[val, random_index]` where:

- `val`: an integer representing `Node.val`

• `random_index`: the index of the node (range from 0 to `n-1`) that

*hame exact copy bnani hai given linkedlist ki
1st step by ignoring random pointers make a copy*

Copy List with Random Pointers



① simple copy

② random connection
to new LL



Copy List with Random Pointers

unordered_map < N, N > m;

Node* newHead = new Node(head->val)

oldTemp = head->next

newTemp = newHead

m[head] = newHead

while (oldTemp != NULL) {

Node* copyNode = new Node(oldTemp->val)

m[oldTemp] = copyNode;

newTemp->next = copyNode

oldTemp = oldTemp->next

newTemp = newTemp->next

}

m Pointers

①

②

```
oldTemp = head; newTemp = newHead  
while (oldTemp != NULL) {
```

```
    newTemp->random = m[oldTemp->random]
```

```
    oldTemp = oldTemp->next
```

```
    newTemp = newTemp->next
```

```
}
```

m Pointers

①

②

```
oldTemp = head; newTemp = newHead  
while (oldTemp != NULL) {
```

```
    newTemp->random = m[oldTemp->random]
```

```
    oldTemp = oldTemp->next
```

```
    newTemp = newTemp->next
```

```
}
```

m Pointers

①

②

```
oldTemp = head; newTemp = newHead  
while (oldTemp != NULL) {
```

```
    newTemp->random = m[oldTemp->random]
```

```
    oldTemp = oldTemp->next
```

```
    newTemp = newTemp->next
```

```
}
```

C++ Auto

```
21         return NULL;  
22     }  
23  
24     unordered_map<Node*, Node*> m;  
25  
26     Node* newHead = new Node(head->val);  
27     Node* oldTemp = head->next;  
28     Node* newTemp = newHead;  
29  
30     while (oldTemp != NULL) {  
31         Node* copyNode = new Node(oldTemp->val);  
32  
33         newTemp->next = copyNode;  
34  
35         oldTemp = oldTemp->next;  
36         newTemp = newTemp->next;  
37     }  
38 }  
39 };
```

```
1  /*
2  // Definition for a Node.
3  class Node {
4      int val;
5      Node next;
6      Node random;
7
8      public Node(int val) {
9          this.val = val;
10         this.next = null;
11         this.random = null;
12     }
13 }
14 */
15
```



```

class Solution {
    public Node copyRandomList(Node head) {
        if(head==null)return null;
        HashMap<Node,Node> mp = new HashMap<>();
        //copy the original node without randompointers
        Node newhead = new Node(head.val);
        Node oldtemp = head.next;
        Node newtemp = newhead;
        mp.put(head,newhead);
        while(oldtemp!=null){
            Node copynode = new Node(oldtemp.val);
            //storing values in map
            mp.put(oldtemp,copynode); //corresponding same values saath mein stored hai
            newtemp.next = copynode;
            newtemp=copynode;
            oldtemp = oldtemp.next;
        }
        // copying the randompointers
        newtemp = newhead;
        oldtemp =head;
        while(oldtemp!=null){
            newtemp.random = mp.get(oldtemp.random);
            oldtemp = oldtemp.next;
            newtemp =newtemp.next;
        }
        return newhead;
    }
}

```

[leetcode](#)

Time & Space Complexity

Metric	Value
Time	$O(n)$
Space	$O(n)$ (HashMap)

 If interviewer asks: *"How did you think of this?"*

Say this 

Each node has a random pointer, so I need a way to connect copied nodes correctly.
I first clone every node and store a mapping from original to copied nodes.
In the second pass, I use this map to correctly assign `next` and `random` pointers.